


# State Machines

## TAC 348 - Making Smart Devices

# Learning Objectives

- Understand what a state machine is
- Describe situations where a state machine is useful
- Analyze a real-world situation to create states, transitions, and input
- Implement a state machine in Photon 2 firmware

# What's the main point?

- Devices should work remotely without constant user intervention
- Sensor readings (**inputs**) and situations (**states**) lead to actions (**outputs**)
- Tracking every possibility becomes complicated
- **State machines** help organize and design for these situations

# State Machine

- A state machine (or *finite state machine*) is a mental **model** to help us make decisions
  - It is not just hardware or just software code
- Examine
  - What has happened previously (**state**)
  - What is happening currently (**input**)
- Then generate
  - Next **output**
  - Next **state** (transition)

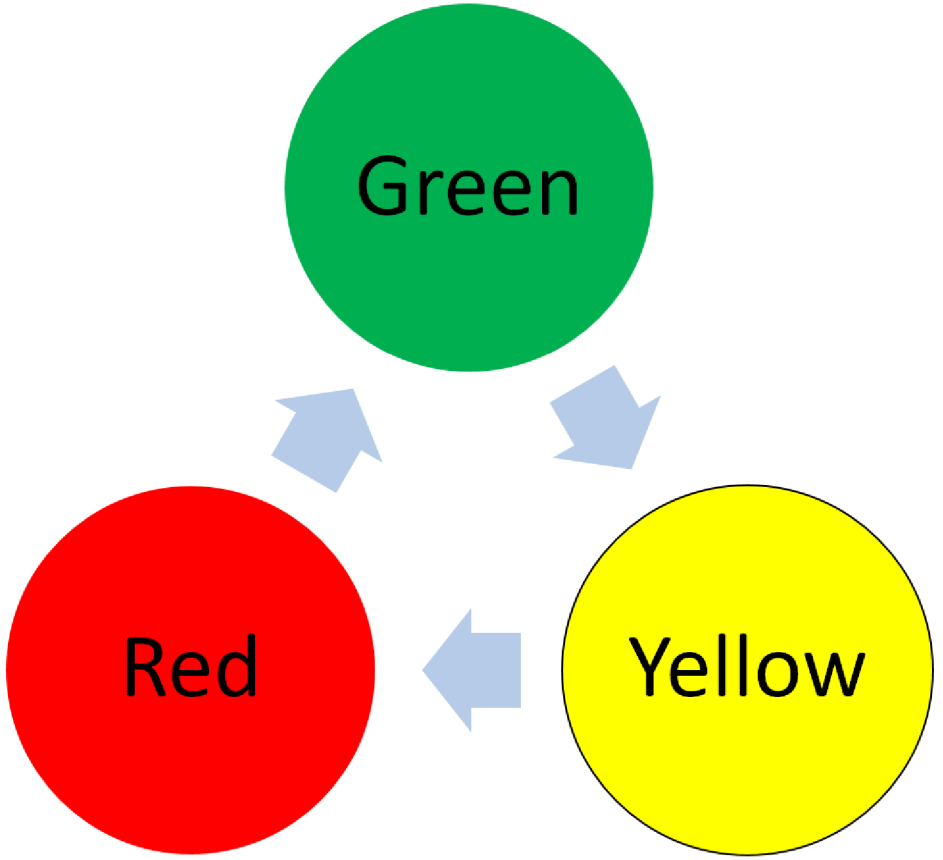
# Example #1
# Stoplight



# Example #1 Stoplight

- Consider a simplified stoplight without any pedestrians
- Lights change after a fixed time
- What states are there?
- How do we transition to new states?

# Example #1
# Stoplight Diagram



# Example #1 Stoplight

| State | Output (Stoplight Color) |
| --- | --- |
| Traffic Flows | Green |
| Traffic Stopping | Yellow |
| Traffic Stopped | Red |

# Example #2 Boba Vending Machine



# Example #2 Boba Vending Machine Diagram

- Similar to soft drink vending machine
- What states are there?
- How do we transition to new states?

# Boba Vending Machine: Transitions

| State | Input | Next State |
|---|---|---|
| Idle | None | Idle |
| Idle | Start button | Display choices |
| Display choices | Choice available | Process credit card |
| Display choices | Choice unavailable | Display choices |
| Display choices | Cancel | Idle |
| Process credit card | Card valid | Dispense boba |
| Process credit card | Card invalid | Process credit card |
| Dispense boba | Boba taken | Idle |
| Dispense boba | Boba remains | Dispense boba |

# State Machines in Photon 2

- In order to implement this in our firmware, what elements / functions do we need to create?
  - inputs
  - outputs
  - state
  - state transition logic
- How could we implement these in firmware?

# State Machines in Photon 2

- inputs (e.g. `millis()` timer, `digitalRead`, `analogRead`)
- outputs (e.g. LED, speaker, OLED screen)
- state (create `enum` to track current state and next state)
- state transition logic
  - typically a function with conditional logic like `switch` or `if`
  - use current state and any input to determine the next state

# State Transition Logic Example

- Let's look at just one piece of the boba vending machine, namely processing valid and invalid credit cards

| State | Input | Next State |
| --- | --- | --- |
| Process credit card | Card valid | Dispense boba |
| Process credit card | Card invalid | Process credit card |

# State Transition Logic Example

- Input: credit card valid or invalid
  - global variable `bool cardValid`
- States: describe all possible states and track current state
  - `enum State {...}`
  - `State currentState;`

```
//Global variables
bool cardValid;   //is customer card valid
enum State {IDLE, DISPLAY, PROCESS, DISPENSE }; //all possible states
State currentState;   //state the machine is in right now
```

# State Transition Logic Example

- State transition: conditional logic to move to next state based on `currentState` and `cardValid`

```
void updateNextState() {
   switch(currentState) {
      case PROCESS:                    //current state is processing credit card
         if (cardValid == true) {      //credit cart is valid so start dispensing
            currentState = DISPENSE;
         }
         //if cardValid == false, remain in PROCESS state
         //no else block is needed
   }
}
```

# Credits

- Bill Siever - CS132 FSM (Washington University at St. Louis)
- Bill Siever - CS132 Delta Timing (Washington University at St. Louis)
- USC EE109 State Machines
- Photo by Michael Olsen on Unsplash
- Photo by Free To Use Sounds on Unsplash
- Photo by NON on Unsplash